Slides:
tinyurl.com
/e6ca822n

You Don't Need an RTOS

Nathan Jones

Slides:
tinyurl.com
/e6ca822n

I want you to consider replacing these...

Slides:
tinyurl.com
/e6ca822n

...with this.


```
while(1)
{
    ...
}
```

Slides:
tinyurl.com
/e6ca822n

Agenda

- Setting the stage
  - What is a scheduler and why do I care?
  - The players: RTOS vs the "Superduperloop"
  - Critical assumptions
- The problem with preemption
- Analyzing response time
- Concluding
  - Other things to consider
  - Comparing an RTOS and the "Superduperloop"

Something we give up (with an RTOS)

Something we gain (with an RTOS)

Slides:
tinyurl.com
/e6ca822n

Setting the Stage

Slides:
tinyurl.com
/e6ca822n

Tasks and Deadlines

Scanner button pressed? → Read barcode and display price

Home sensor reached? → Turn off motor

https://tinyurl.com/2r9uu8uy | https://tinyurl.com/23pd855w

Slides:
tinyurl.com
/e6ca822n

The Scheduler

Released / Triggered

Read barcode and display price

Play sound

Open cash drawer

Time 0 5 10

Slides:
tinyurl.com
/e6ca822n

The Scheduler

Released / Triggered

Read barcode and display price

Play sound

Open cash drawer

Read barcode and display price

Play sound

Open cash drawer

Scheduled / Invoked

Time 0 5 10

Slides:
tinyurl.com
/e6ca822n

The Scheduler

Released / Triggered

Read barcode and display price

Play sound

Open cash drawer

Read barcode and display price

Play sound

Open cash drawer

Period / Fastest "inter-arrival" time

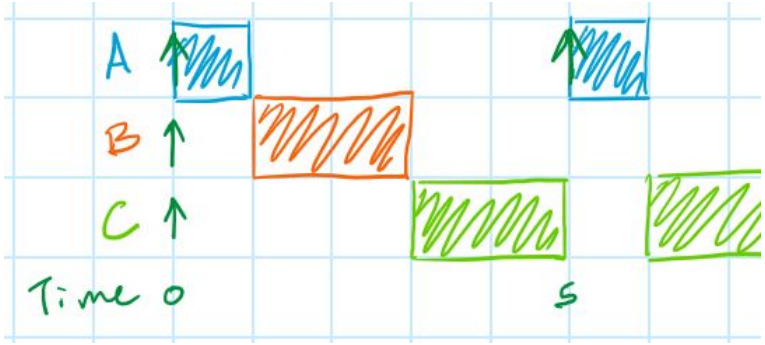
Scheduled / Invoked

Deadlines

Time 0 5 10

Slides:
tinyurl.com
/e6ca822n

(Preemptive) RTOS



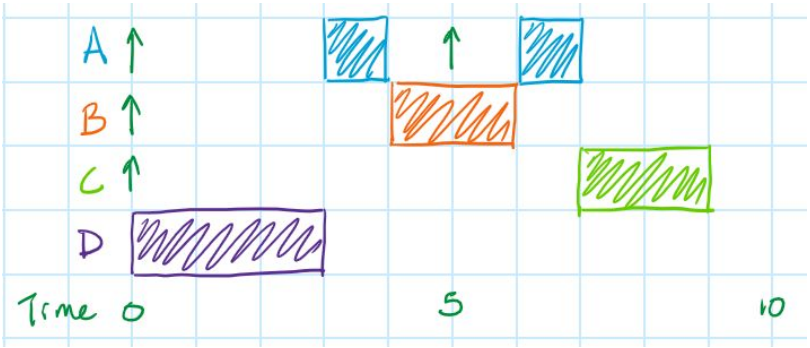

```
void main(void) {
    taskCreate(TaskA, 0);   // Priority 0 (highest)
    taskCreate(TaskB, 1);   // Priority 1
    taskCreate(TaskC, 2);   // Priority 2
    startScheduler();       // No return
}

void TaskA(void) {
    while(1) {
        // Task A
    }
}
...
```

```
osDelay(500);
osThreadFlagsSet(TaskB_id, 0x0100U);
xSemaphoreTake(mySemaphore, (TickType_t) 10);
xQueueSend(myQueue, (void *)&val, portMAX_DELAY);
```

Slides:
tinyurl.com
/e6ca822n

The Superloop or... The "Superduperloop"!

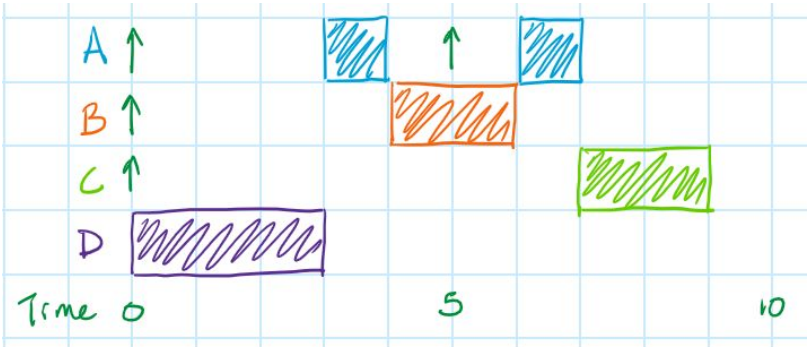
```
enum { A_NUM, B_NUM, NUM_TASKS };
const int A_MASK = (1<<A_NUM);
const int B_MASK = (1<<B_NUM);
volatile int g_tasks = 0;
void main(void) {
    while(1) {
        if     ( g_tasks & A_MASK ) TaskA();
        else if( g_tasks & B_MASK ) TaskB();
        else     /* Go to sleep */;
    }
}
```



Slides:
tinyurl.com
/e6ca822n

The Superloop or… The "Superduperloop"!

```
enum { A_NUM, B_NUM, NUM_TASKS };
const int A_MASK = (1<<A_NUM);
const int B_MASK = (1<<B_NUM);
volatile int g_tasks = 0;
void main(void) {
    while(1) {
        if     ( g_tasks & A_MASK ) TaskA();
        else if( g_tasks & B_MASK ) TaskB();
        else     /* Go to sleep */;
    }
}
```



Establishes **priorities**

Slides:
tinyurl.com
/e6ca822n

Critical Assumptions

1. All tasks are periodic (or, at least, have a minimum "inter-arrival period")
2. All tasks' deadlines are less than or equal to their periods
3. All tasks are completely independent
4. It takes zero time for the scheduler to switch between tasks
5. Tasks are being executed on a single-core processor

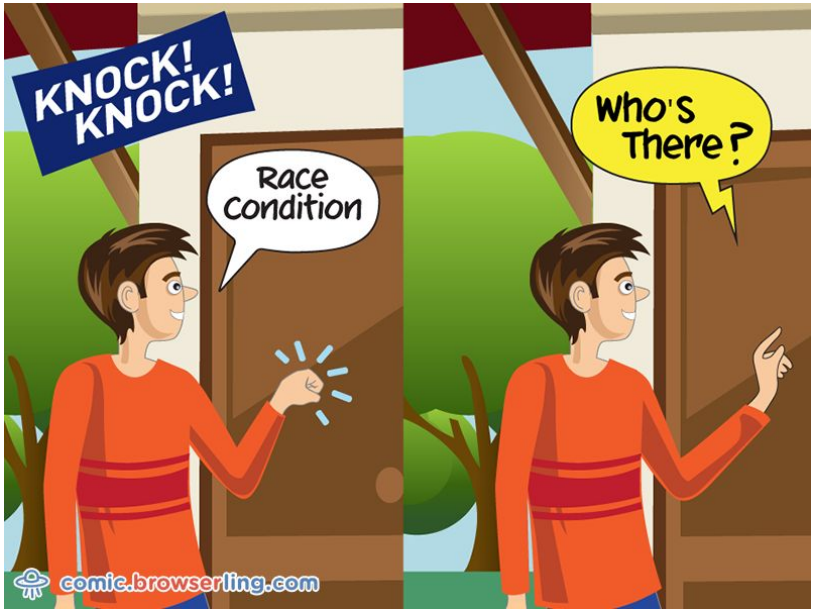
**None (or almost none) of these are 100% true all of the time.

Slides:
tinyurl.com
/e6ca822n

The Problem with Preemption

Slides:
tinyurl.com
/e6ca822n

Race conditions!



Amir Shevat 
@ashevat

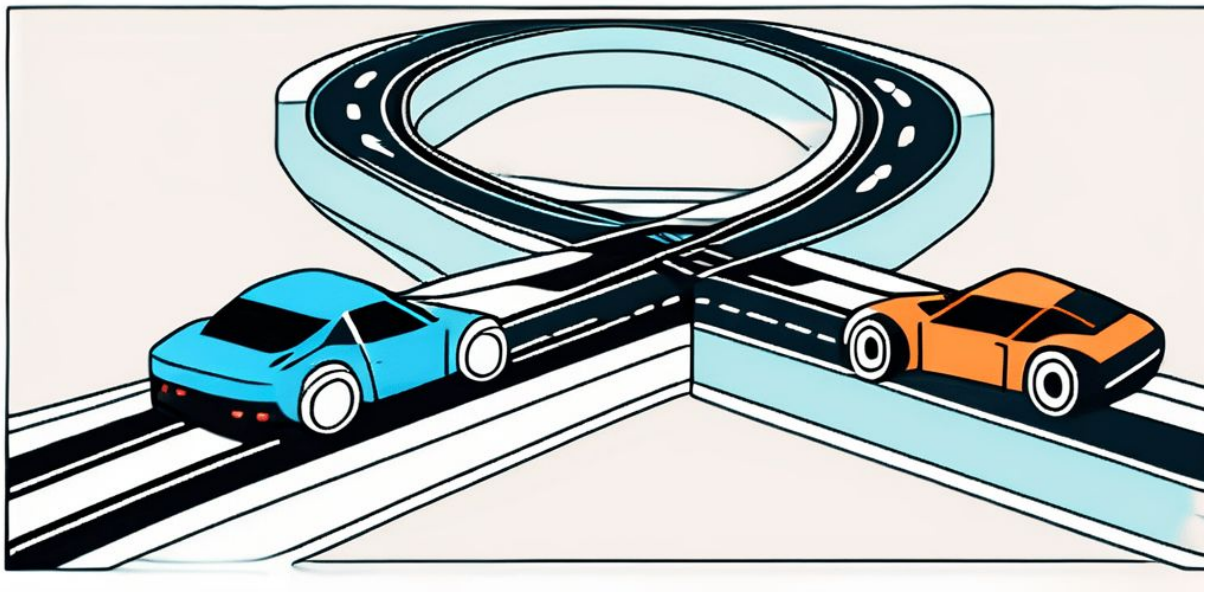
A programmer had a problem. He thought to himself, "I know, I'll solve it with threads!". has Now problems. two he

10:00 PM · 16 Jan 21 · Twitter for iPhone

https://tinyurl.com/bdfw8jyx | https://x.com/ashevat/status/1350548449364623361?lang=en

Slides:
tinyurl.com
/e6ca822n

Race conditions!



https://bluegoatcyber.com/wp-content/uploads/2024/02/img_65cbfaed250bb.png

Slides:
tinyurl.com
/e6ca822n

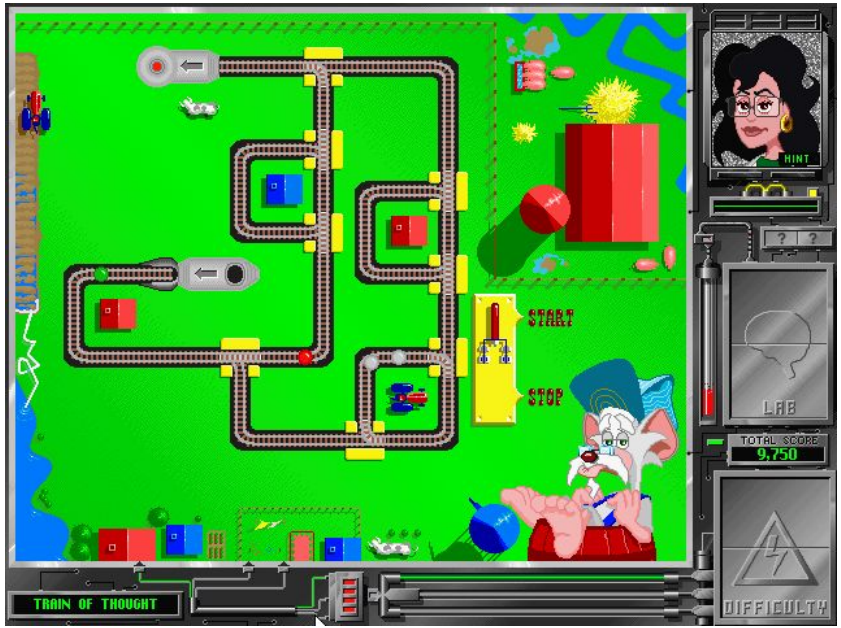
Fun fact!

"Studies show that a little over 1 percent of population can effectively multitask. Odds are you aren't in that 1 percent. That's just how odds work."

-*Peak Performance*, by Brad Stulberg and Steve Magness

Slides:
tinyurl.com
/e6ca822n

Race conditions (1995-style)



https://tinyurl.com/4xv95mfm (from "The Lost Mind of Dr. Brain")

Slides:
tinyurl.com
/e6ca822n

Race conditions are...

hard to keep out of your code

and

hard to debug once they're there.

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

```
transfer1 (amount, account_from, account_to)
{
     if (account_from.balance < amount) return NOPE;
     account_to.balance += amount;
     account_from.balance -= amount;
     return YEP;
}
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

```
transfer1 (amount, account_from, account_to)
{
     if (account_from.balance < amount) return NOPE;
     ldr r4, [account_to.balance];          // account_to.balance += amount;
     add r4, r3                             // amount in r3
     str r4, [account_to.balance];
     account_from.balance -= amount;
     return YEP;
}
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

r3	150
r4	xxxx
account_to.balance	200

Thread B

```
transfer1 (amount, account_from, account_to)

    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
}
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

r3	150
r4	200
account_to.balance	200

```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
}
```

Thread B

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

r3	150
r4	350
account_to.balance	200

```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
}
```

Thread B

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Thread A	
r3	75
r4	xxxx
account_to.balance	200

Thread B	
r3	150
r4	350
account_to.balance	200

```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
}
```

Thread A

Thread B

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Thread A	
r3	75
r4	275
account_to.balance	275

Thread B	
r3	150
r4	350
account_to.balance	200

```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
}
```

Thread B

Thread A

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

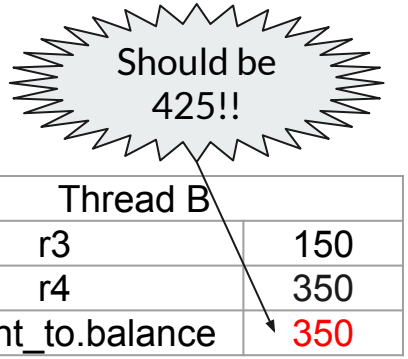
Thread B	
r3	150
r4	350
account_to.balance	275

```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
}
```

Thread B

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?




```
transfer1 (amount, account_from, account_to)
{
    if (account_from.balance < amount) return NOPE;
    ldr r4, [account_to.balance];          // account_to.balance += amount;
    add r4, r3                             // amount in r3
    str r4, [account_to.balance];
    account_from.balance -= amount;
    return YEP;
```

Thread B

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

```
if (!initialized)
{
     object = create();
     initialized = true;
}
... object ...
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Thread B

```
if (!initialized)
{
    object = create();
    initialized = true;
}
... object ...
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Number of objects	1

```
if (!initialized)
{
    object = create();
    initialized = true;
}
... object ...
```

Thread B

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Number of objects	1

Thread A

Thread B

```
if (!initialized)
{
    object = create();
    initialized = true;
}
... object ...
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Number of objects	2

```
if (!initialized)
{
    object = create();
    initialized = true;
}
... object ...
```

Thread B

Thread A

Slides:
tinyurl.com
/e6ca822n

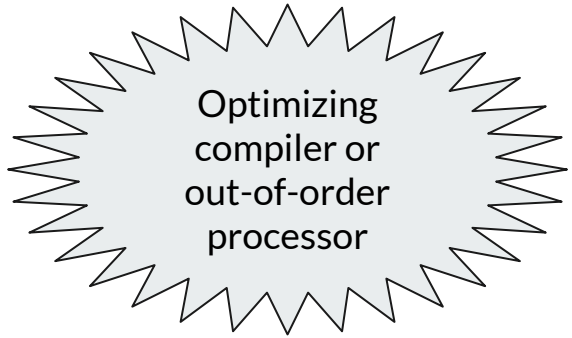
Where's the race condition?

```
if (!initialized)
{
     lock();
     if (!initialized)
     {
          object = create();
          initialized = true;
     }
     unlock();
}
... object ...
```

Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

```
if (!initialized)
{
     lock();
     if (!initialized)
     {
          initialized = true;
          object = create();
     }
     unlock();
}
... object ...
```

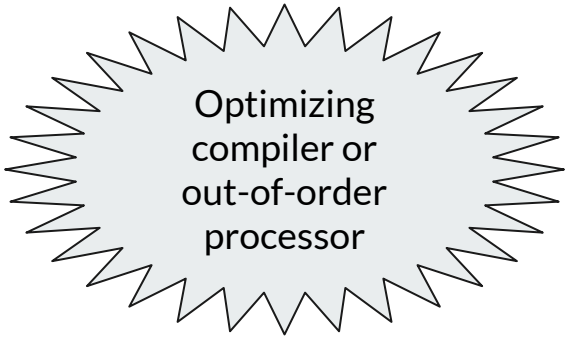


Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Thread B

```
if (!initialized)

    lock();
    if (!initialized)
    {
        initialized = true;
        object = create();
    }
    unlock();
}
... object ...
```

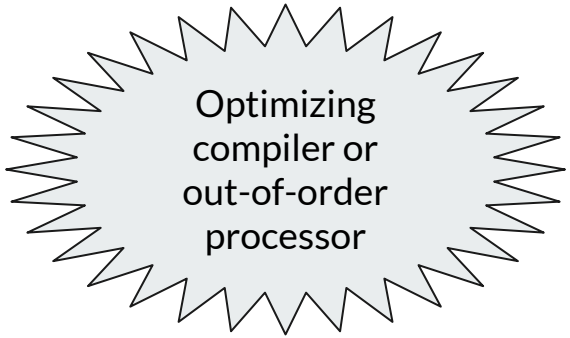


Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

```
if (!initialized)
{
     lock();
     if (!initialized)
     {
          initialized = true;
          object = create();
     }
     unlock();
}
... object ...
```

Thread B



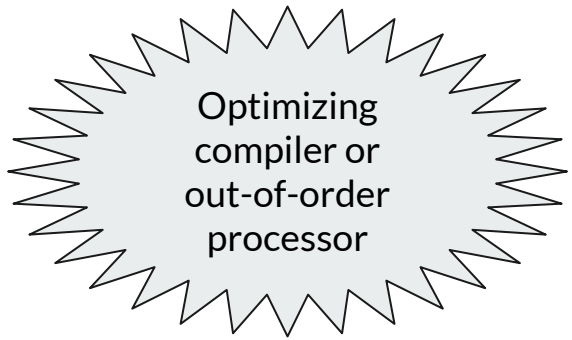
Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

Thread A

```
if (!initialized)
{
     lock();
     if (!initialized)
     {
          initialized = true;
          object = create();
     }
     unlock();
}
... object ...
```

Thread B



Slides:
tinyurl.com
/e6ca822n

Where's the race condition?

```
if (!initialized)
{
     lock();
     if (!initialized)
     {
          initialized = true;
          object = create();
     }
     unlock();
}
... object ...
```

Thread B

Thread A

Optimizing compiler or out-of-order processor

Object not actually created yet!!

Slides:
tinyurl.com
/e6ca822n

Identifying race conditions

No automated tools!

Go through **every line of code** for your **entire system** and ask:

- "Could anything bad happen if one or more higher priority tasks ran in between these two machine instructions?"
- "Could anything bad happen if this task runs in between any two machine instructions in a lower priority task?"

Slides:
tinyurl.com
/e6ca822n

Preventing race conditions

- Use atomic operations on shared data
- Disable interrupts
- Protect shared resources with mutexes
- Control the reordering of code with memory barriers
- Set rendezvous points using semaphores/flags

Slides:
tinyurl.com
/e6ca822n

Preventing race conditions

- Use atomic operations on shared data
- **Disable interrupts** ⟶
- Protect shared resources with mutexes
- Control the reordering of code with memory barriers
- Set rendezvous points using semaphores/flags

"You know what? Maybe this RTOS should have been a superloop…"

https://tinyurl.com/3pve76ns

Slides:
tinyurl.com
/e6ca822n

Preventing race conditions

- Use atomic operations on shared data
- Disable interrupts
- **Protect shared resources with mutexes**
- Control the reordering of code with memory barriers
- Set rendezvous points using semaphores/flags

Deadlock

Figure - 1

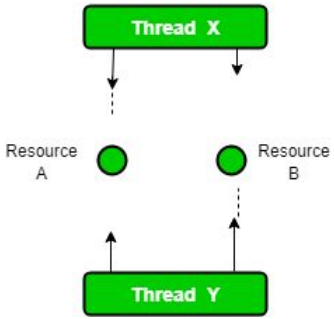
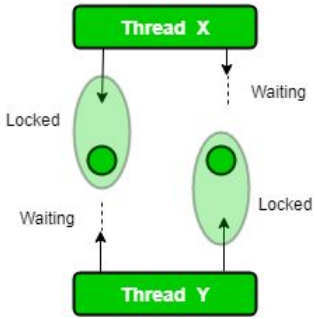


Figure - 2



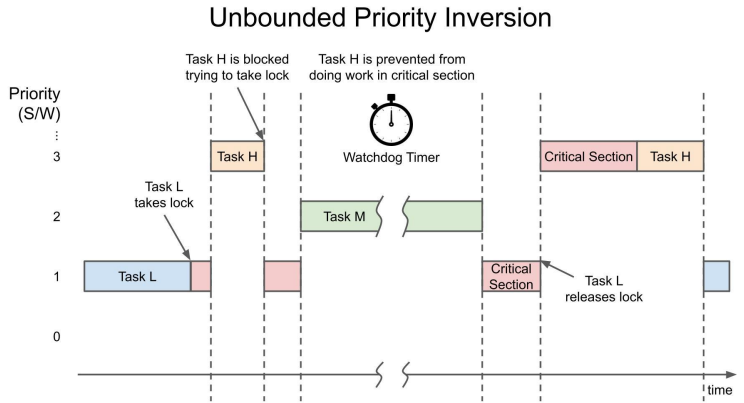
https://www.geeksforgeeks.org/deadlock-in-java-multithreading/

Slides:
tinyurl.com
/e6ca822n

Preventing race conditions

- Use atomic operations on shared data
- Disable interrupts
- **Protect shared resources with mutexes**
- Control the reordering of code with memory barriers
- Set rendezvous points using semaphores/flags

Priority Inversion



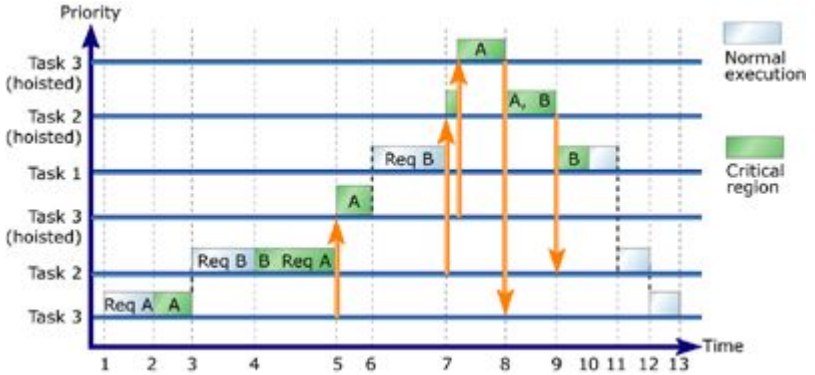
https://tinyurl.com/mrx3xven

Slides:
tinyurl.com
/e6ca822n

Preventing race conditions

- Use atomic operations on shared data
- Disable interrupts
- **Protect shared resources with mutexes**
- Control the reordering of code with memory barriers
- Set rendezvous points using semaphores/flags

Priority Inversion



https://www.embedded.com/how-to-use-priority-inheritance/

Slides:
tinyurl.com
/e6ca822n

Preventing race conditions

- Use atomic operations on shared data
- Disable interrupts
- **Protect shared resources with mutexes** ⟶
- Control the reordering of code with memory barriers
- Set rendezvous points using semaphores/flags

https://tinyurl.com/2u4byr32

Slides:
tinyurl.com
/e6ca822n

Race conditions...

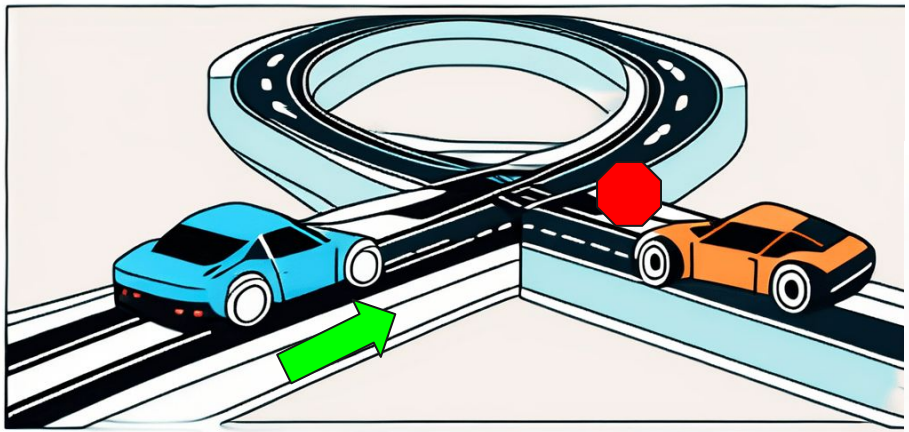
are hard to keep out of your code,

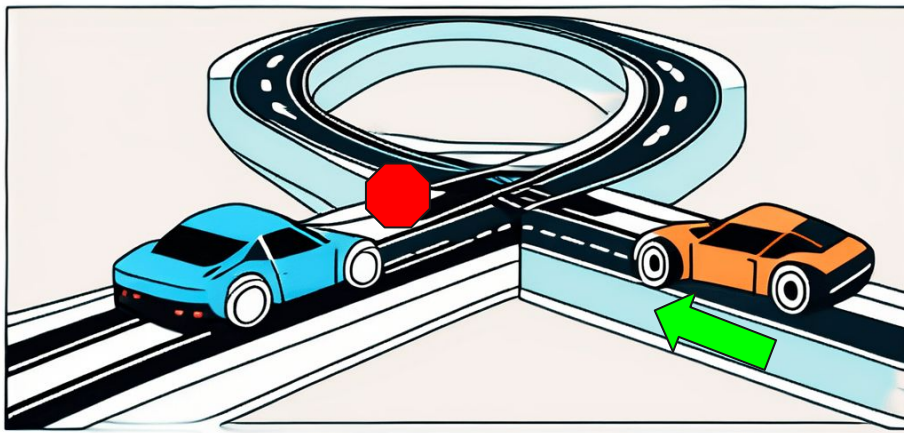
they're hard to debug once they're there,

and their solutions often cause more problems.

Slides:
tinyurl.com
/e6ca822n

Single-threaded code effectively eliminates races





https://bluegoatcyber.com/wp-content/uploads/2024/02/img_65cbfaed250bb.png

Slides:
tinyurl.com
/e6ca822n

What does this all mean?

Preemption should **only ever** be introduced in a system that **absolutely requires** it for the system to be schedulable.

Once introduced, **extreme care** (read: extensive and specific coding/review guidelines) should be taken to ensure there are no race conditions.

https://tinyurl.com/3pk96p8u

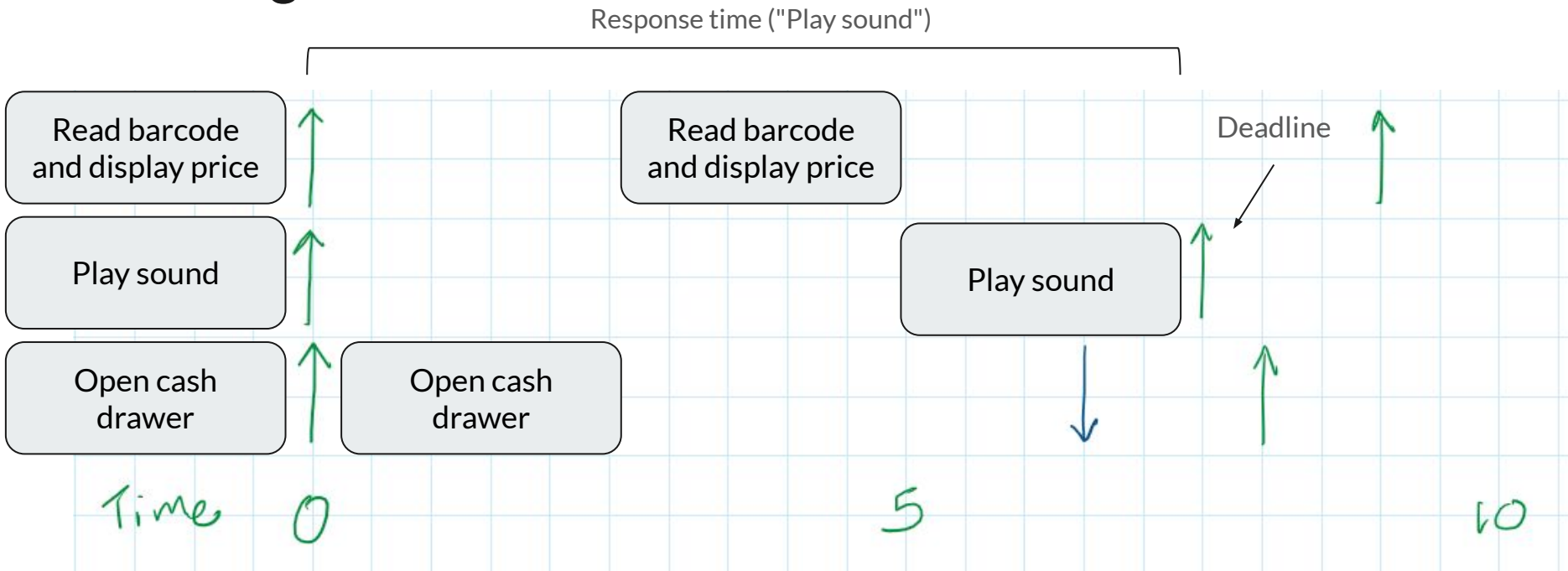
Slides:
tinyurl.com
/e6ca822n

Analyzing Response Time

Slides:
tinyurl.com
/e6ca822n

IF (Worst-case response time$_{Task}$ < Deadline$_{Task}$)
THEN Task **IS** schedulable
ELSE Task is **NOT** schedluable

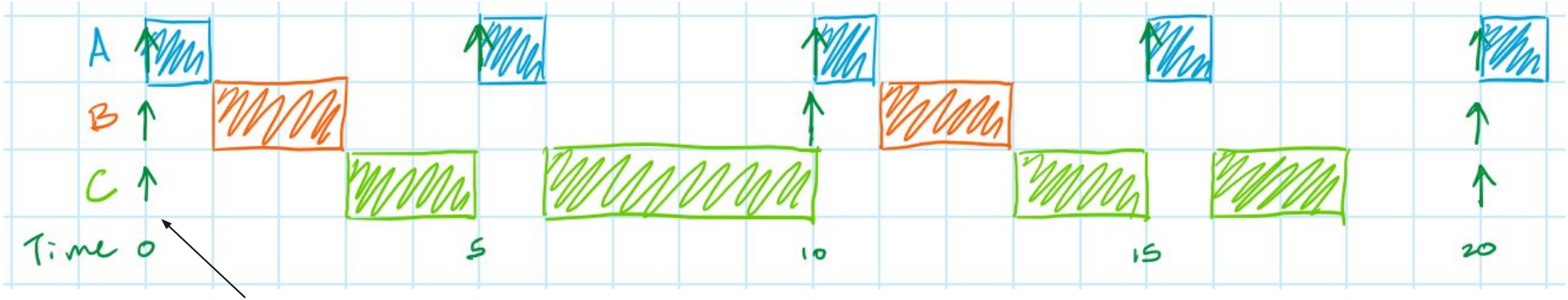
The Big Picture



Slides:
tinyurl.com
/e6ca822n

Worst-case response time: RTOS

Task	Priority	WCET*	Period
A	0	1	5
B	1	2	10
C	2	10	20



"Critical instant"

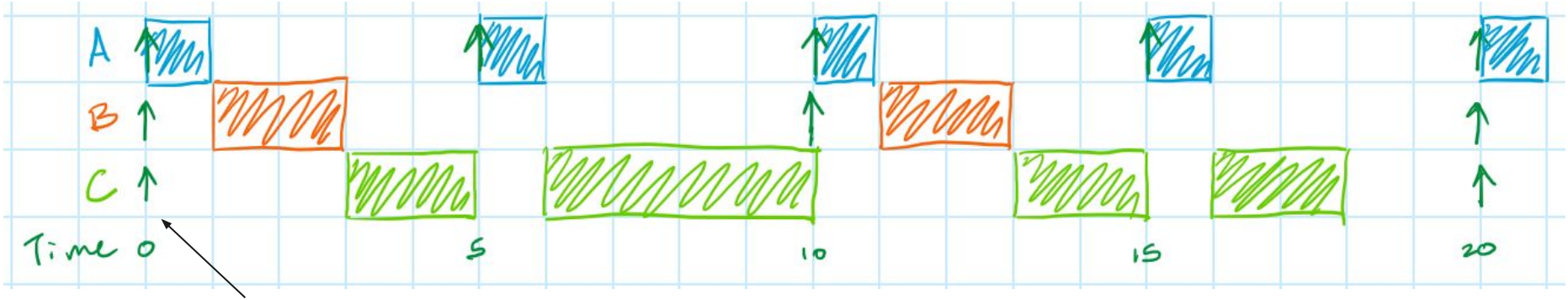
"WCET": Worst-case execution time

Slides:
tinyurl.com
/e6ca822n

Worst-case response time: RTOS

Task	Priority	WCET*	Period
A	0	1	5
B	1	2	10
C	2	10	20

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$



"Critical instant"

"WCET": Worst-case execution time

Slides:
tinyurl.com
/e6ca822n

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Worst-case response time: RTOS

Task	Priority	WCET	Period
A	0	1	5
B	1	2	10
C	2	10	20

$$10 \stackrel{?}{=} \left\lceil \frac{10}{5} \right\rceil \cdot 1 + \left\lceil \frac{10}{10} \right\rceil \cdot 2 + 10 = 2 \cdot 1 + 1 \cdot 2 + 10 = 14$$

Slides:
tinyurl.com
/e6ca822n

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Worst-case response time: RTOS

Task	Priority	WCET	Period
A	0	1	5
B	1	2	10
C	2	10	20

$$10 \stackrel{?}{=} \left\lceil \frac{10}{5} \right\rceil \cdot 1 + \left\lceil \frac{10}{10} \right\rceil \cdot 2 + 10 = 2 \cdot 1 + 1 \cdot 2 + 10 = 14$$

$$14 \stackrel{?}{=} \left\lceil \frac{14}{5} \right\rceil \cdot 1 + \left\lceil \frac{14}{10} \right\rceil \cdot 2 + 10 = 3 \cdot 1 + 2 \cdot 2 + 10 = 17$$

Slides:
tinyurl.com
/e6ca822n

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Worst-case response time: RTOS

Task	Priority	WCET	Period
A	0	1	5
B	1	2	10
C	2	10	20

$$10 \stackrel{?}{=} \left\lceil \frac{10}{5} \right\rceil \cdot 1 + \left\lceil \frac{10}{10} \right\rceil \cdot 2 + 10 = 2 \cdot 1 + 1 \cdot 2 + 10 = 14$$

$$14 \stackrel{?}{=} \left\lceil \frac{14}{5} \right\rceil \cdot 1 + \left\lceil \frac{14}{10} \right\rceil \cdot 2 + 10 = 3 \cdot 1 + 2 \cdot 2 + 10 = 17$$

$$17 \stackrel{?}{=} \left\lceil \frac{17}{5} \right\rceil \cdot 1 + \left\lceil \frac{17}{10} \right\rceil \cdot 2 + 10 = 4 \cdot 1 + 2 \cdot 2 + 10 = 18$$

Slides:
tinyurl.com
/e6ca822n

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Worst-case response time: RTOS

Task	Priority	WCET	Period
A	0	1	5
B	1	2	10
C	2	10	20

$$10 \stackrel{?}{=} \left\lceil \frac{10}{5} \right\rceil \cdot 1 + \left\lceil \frac{10}{10} \right\rceil \cdot 2 + 10 = 2 \cdot 1 + 1 \cdot 2 + 10 = 14$$

$$14 \stackrel{?}{=} \left\lceil \frac{14}{5} \right\rceil \cdot 1 + \left\lceil \frac{14}{10} \right\rceil \cdot 2 + 10 = 3 \cdot 1 + 2 \cdot 2 + 10 = 17$$

$$17 \stackrel{?}{=} \left\lceil \frac{17}{5} \right\rceil \cdot 1 + \left\lceil \frac{17}{10} \right\rceil \cdot 2 + 10 = 4 \cdot 1 + 2 \cdot 2 + 10 = 18$$

$$18 \stackrel{?}{=} \left\lceil \frac{18}{5} \right\rceil \cdot 1 + \left\lceil \frac{18}{10} \right\rceil \cdot 2 + 10 = 4 \cdot 1 + 2 \cdot 2 + 10 = 18$$

Slides:
tinyurl.com
/e6ca822n

Task	Priority	WCET	Period
A	0	1	5
B	1	2	6
C	2	3	7

Your turn!

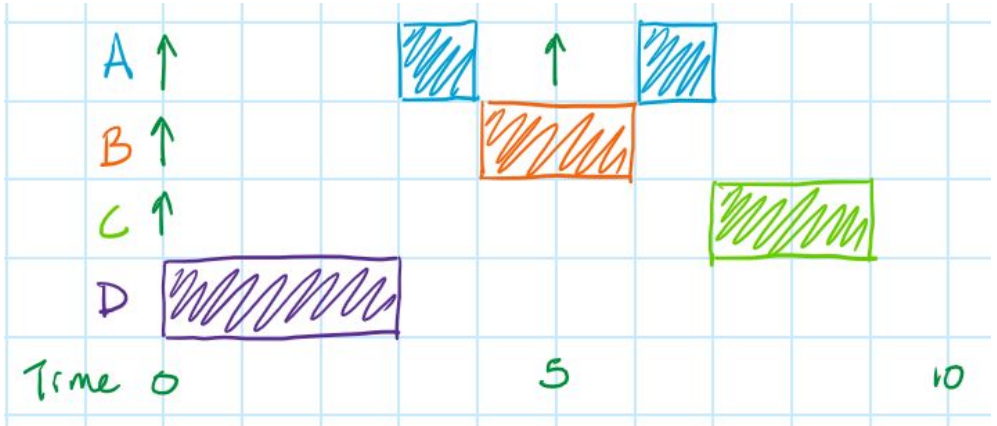
- What are the response times for the tasks above (using an RTOS)?
- Is the system schedulable?

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Slides:
tinyurl.com
/e6ca822n

Worst-case response time: "Superduperloop"

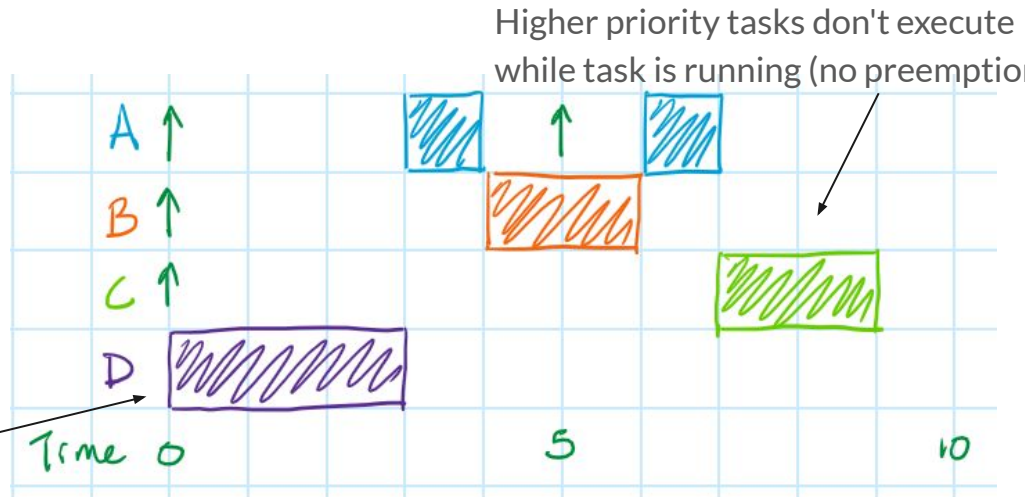
Task	Priority	WCET	Period
A	0	1	5
B	1	2	15
C	2	2	15
D	3	3	20



Slides:
tinyurl.com
/e6ca822n

Worst-case response time: "Superduperloop"

Task	Priority	WCET	Period
A	0	1	5
B	1	2	15
C	2	2	15
D	3	3	20



Critical instant;
LONGEST, LOWER priority task runs *riiight* before

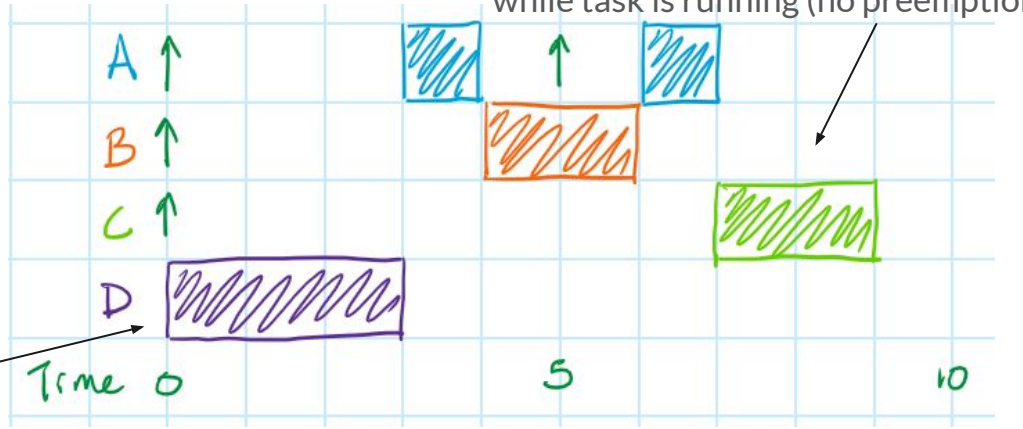
Slides:
tinyurl.com
/e6ca822n

Worst-case response time: "Superduperloop"

$$WCRT_j = max(WCET_{i \in i>j}) + \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j - WCET_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Task	Priority	WCET	Period
A	0	1	5
B	1	2	15
C	2	2	15
D	3	3	20

Higher priority tasks don't execute while task is running (no preemption)



Critical instant;

LONGEST, LOWER priority task runs *riiight* before

Slides:
tinyurl.com
/e6ca822n

Task	Priority	WCET	Period
A	0	1	5
B	1	2	6
C	2	3	7

Your turn!

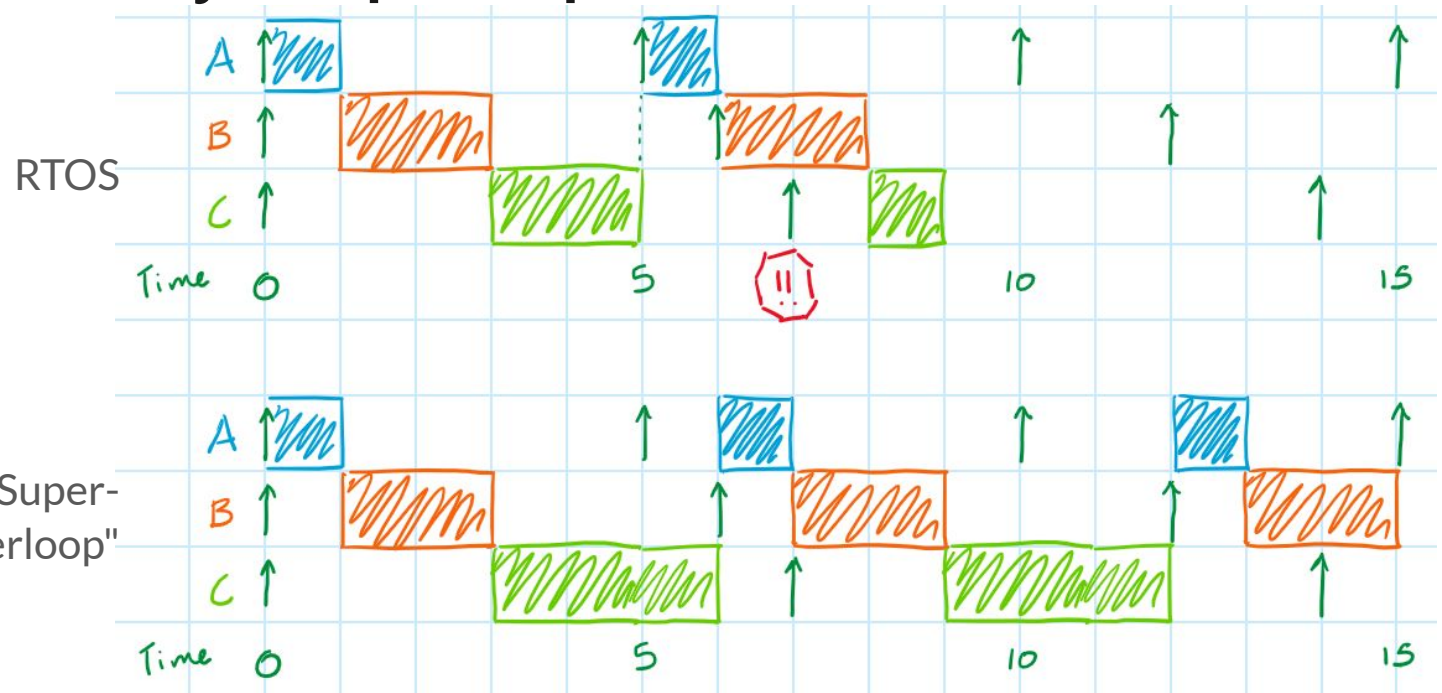
- What are the response times now, using the "Superduperloop"?
- Is the system schedulable?

$$WCRT_j = max(WCET_{i \in i>j}) + \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j - WCET_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

Slides:
tinyurl.com
/e6ca822n

Task	Priority	WCET	Period	WCRT (RTOS)	WCRT (Superduperloop)
A	0	1	5	1	4
B	1	2	6	3	6
C	2	3	7	9	6

The day a Superloop beat an RTOS!!



Slides:
tinyurl.com
/e6ca822n

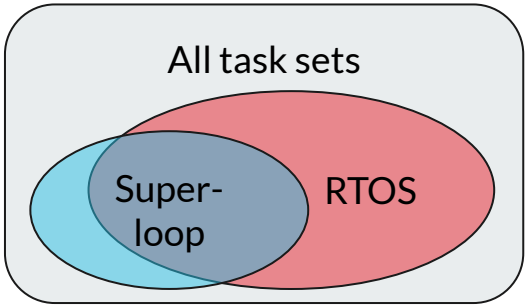
The day a Superloop beat an RTOS!!

$$WCRT_j = \underbrace{\sum_{i=0}^{j-1}\left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i\right)} + WCET_j$$

NO equivalence!!

$$WCRT_j = max(WCET_{i \in i>j}) + \sum_{i=0}^{j-1}\left(\left\lceil \frac{WCRT_j - WCET_j}{P_i} \right\rceil \cdot WCET_i\right) + WCET_j$$

Slides:
tinyurl.com
/e6ca822n



The day a Superloop beat an RTOS!!

$$WCRT_j = \underbrace{\sum_{i=0}^{j-1}\left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i\right)} + WCET_j$$

NO equivalence!!

$$WCRT_j = max(WCET_{i \in i>j}) + \sum_{i=0}^{j-1}\left(\left\lceil \frac{WCRT_j - WCET_j}{P_i} \right\rceil \cdot WCET_i\right) + WCET_j$$

Slides:
tinyurl.com
/e6ca822n

Concluding

Slides:
tinyurl.com
/e6ca822n

RTOS vs "Superduperloop"

X	Better WCRT for high-priority tasks	
	Equitable WCRT	X
	No race conditions	X
X	Includes thread flags, queues, semaphores, etc*	
	Small stack requirements	X
	Doesn't require porting	X
	Small code size / Simple	X

Slides:
tinyurl.com
/e6ca822n

Other Things to Consider

"You Don't Need an RTOS" on

- Measuring a task's worst-case execution time
- Improving a task's worst-case execution time
- Improving a task set's schedulability
- Assigning task priorities
- Open-source/Commercial superloops
- Inter-process communication (i.e. thread flags, event flags, mailboxes/queues, etc)
  - Schedulability of a task with deadline *greater than* its period
- Alternative non-preemptive schedulers
  - Time-triggered scheduler
  - Dispatch queue

https://www.embeddedrelated.com/showarticle/1636.php

Slides:
tinyurl.com
/e6ca822n

What lingering concerns or questions do you still have?

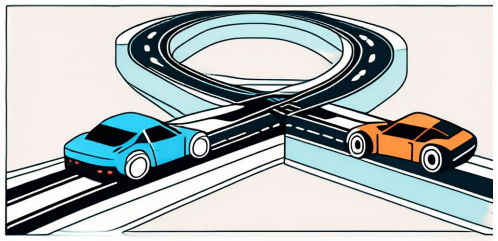
Slides:
tinyurl.com
/e6ca822n

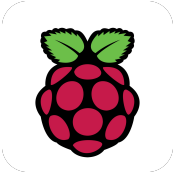
"You Don't Need an RTOS" on Embedded RELATED.com

Summary

$$WCRT_j = max(WCET_{i \in i>j}) + \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j - WCET_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

RTOS vs "Superduperloop"



Like  vs 

$$WCRT_j = \sum_{i=0}^{j-1} \left(\left\lceil \frac{WCRT_j}{P_i} \right\rceil \cdot WCET_i \right) + WCET_j$$

https://tinyurl.com/vmrzppbk | https://tinyurl.com/4jz6mbm3